AI Interview Trainer

Complete Project Documentation

A web application that lets users practice job interviews with an AI-powered interviewer.

AI generates questions, evaluates answers, and returns scores plus feedback

(strengths, weaknesses, suggestions, and a model answer).

Technology stack: Python 3 + FastAPI + SQLAlchemy + Google Gemini API

Frontend: HTML5, CSS3, Vanilla JavaScript

Database: SQLite (in active use) with MySQL as the intended production database

Documentation date: 06 September 2026

Project status: working development build (version 1.0.0)

Local URL: http://127.0.0.1:8000

This document describes the application as implemented in the source code. It does not contain any secret values (API keys and passwords are redacted).

Table of Contents

1. Cover Page - Project at a Glance **6**

1.1 Project purpose . . . 6

1.2 Documentation contents . . . 6

2. Executive Summary **7**

2.1 What the project is . . . 7

2.2 The problem it solves . . . 7

2.3 Who uses it . . . 7

2.4 Major features . . . 7

2.5 Overall workflow . . . 8

3. Technology Stack **9**

4. Project Directory **10**

File roles (backend): . . . 11

File roles (frontend): . . . 13

Database + configuration + support files: . . . 14

5. System Architecture **15**

5.1 How the parts communicate . . . 15

5.2 Why this design . . . 16

6. Backend Architecture **17**

6.1 Application entry point . . . 17

6.2 Configuration . . . 17

6.3 Database access . . . 17

6.4 Authentication . . . 17

6.5 Routes, services, schemas, models . . . 17

6.6 Error handling, logging, CORS . . . 17

6.7 API table . . . 18

6.8 HTML page routes served by the backend . . . 18

7. Frontend Architecture **19**

7.1 Pages . . . 19

7.2 Shared HTTP client (api.js) . . . 19

7.3 Interview UI (interview.js) . . . 19
7.4 Results UI (result.js) . . . 20
7.5 Loading, error and empty states . . . 20
7.6 Admin UI . . . 20
8. Database **21**
8.1 Database type and file . . . 21
8.2 Actual tables (verified from interview.db) . . . 21
8.3 Relationship diagram (matches ORM models) . . . 21
8.4 Important queries / CRUD . . . 21
9. AI Integration **23**
9.1 Provider, model, environment variables . . . 23
9.2 Client initialization . . . 23
9.3 Prompt generation . . . 23
9.4 Request and retry/rate-limit handling . . . 23
9.5 Response handling and JSON parsing . . . 23
9.6 Fallback behavior . . . 24
9.7 AI flow diagram . . . 24
10. Interview Flow **25**
11. Question System **26**
11.1 Generation and selection . . . 26
11.2 Roles, interview types, difficulty, count . . . 26
11.3 Storage . . . 26
11.4 MCQ rules and options . . . 26
11.5 Next-question logic . . . 26
12. Difficulty System **27**
13. Timer System **28**
14. Scoring System **29**
14.1 Per-question scoring . . . 29
14.2 Interview totals . . . 29
14.3 Result percentage and grading . . . 29
14.4 Strengths / weaknesses / suggestions . . . 29

14.5 Timer penalties . . . 29
15. Admin System **30**
16. Authentication & Security **31**
16.1 Registration and login . . . 31
16.2 Password safety . . . 31
16.3 Frontend token handling . . . 31
16.4 Other measures and notes . . . 31
16.5 Security concerns to note . . . 31
17. Error Handling **33**
17.1 Server-side error surfacing . . . 34
18. Logging & Debugging **35**
18.1 Where logs live . . . 35
18.2 Debugging guide . . . 35
19. Configuration **36**
19.1 Environment variables . . . 36
19.2 Host / port / development settings . . . 36
20. Dependencies **37**
21. How to Run Manually on Windows **38**
21.1 1) Check Python . . . 38
21.2 2) Activate the virtual environment . . . 38
21.3 3) Install dependencies . . . 38
21.4 4) Configure environment . . . 38
21.5 5) Start the backend . . . 38
21.6 6) Open the browser . . . 38
21.7 7) Stop the server . . . 38
21.8 8) Check the port / 9) Restart . . . 38
22. Manual Testing Guide **39**
23. Terminal Command Reference **41**
24. Current Bugs & Recommendations **42**
24.1 Critical . . . 42

24.2 High 42
24.3 Medium 43
24.4 Low 43
25. Interview Explanation **44**
25.1 30-second explanation 44
25.2 2-minute explanation 44
25.3 5-minute technical explanation 44
25.4 Architecture explanation 44
26. Project Interview Questions **45**
27. Final Project Summary **47**

1. Cover Page - Project at a Glance

1.1 Project purpose

The AI Interview Trainer is a web application that simulates a job interview. A user registers, logs in, selects a job role, interview type, difficulty level, and the number of questions. The system then presents AI-generated interview questions. When the user submits an answer, the system uses AI to evaluate it and returns a score (0-10) together with feedback: strengths, weaknesses, improvement suggestions, and a model answer. When the interview is finished, an overall feedback summary is generated and the interview is saved to the database so the user can review history on the dashboard.

1.2 Documentation contents

- Sections 2-4: Executive summary, technology stack, project directory.
- Sections 5-8: System, backend, frontend and database architecture.
- Sections 9-16: AI integration, interview flow, question/difficulty/timer/scoring/admin systems, and security.
- Sections 17-20: Error handling, logging, configuration, dependencies.
- Sections 21-24: Run instructions, manual testing, terminal reference, known bugs.
- Sections 25-27: Interview explanation, project interview questions, final summary.

2. Executive Summary

The **AI Interview Trainer** solves a common problem for students and job seekers: there is not enough low-cost, realistic interview practice. Hired coaches are expensive and fixed question banks do not adapt to the candidate. This application uses an AI service (Google Gemini) to generate questions on demand and to evaluate each written answer instantly, giving the user actionable feedback.

2.1 What the project is

It is a full-stack web application with a vanilla JavaScript frontend, a Python FastAPI backend, and a SQLAlchemy-managed database. The AI layer generates questions, evaluates written answers, and writes overall feedback.

2.2 The problem it solves

- Limited interview practice time before placements/jobs.
- No instant or objective feedback on answers.
- Generic question banks do not match role and difficulty.
- No easy way to track improvement over time.

2.3 Who uses it

Students preparing for campus placement interviews, and job seekers who want to practise technical and HR/behavioral interviews.

2.4 Major features

- User registration and JWT-based login.
- Interview setup: job role, interview type (technical / HR-behavioral / mixed), difficulty (beginner / intermediate / advanced), and question count (5 or 10, schema allows 5-20).
- AI-generated questions: MCQ for easy/intermediate, written questions for hard/intermediate.
- Instant AI evaluation with a score of 0-10, strengths, weaknesses, suggestions and a model answer.
- Overall interview feedback after completion.
- Dashboard with total interviews, completed count, average and highest score, and interview history.
- Results page with question-by-question breakdown.

2.5 Overall workflow

```
User opens application
  |
  v
Register / Login (JWT token stored in browser)
  |
  v
Dashboard -> Start Interview -> choose role / type / difficulty / count
  |
  v
Backend generates a question (AI or local fallback bank)
  |
  v
User answers (MCQ selection or written text)
  |
  v
Backend evaluates (MCQ: correct/incorrect scoring; written: Gemini evaluation)
  |
  v
Score + feedback shown -> Next question or Finish
  |
  v
Overall feedback generated, interview marked completed, saved to database
  |
  v
Results page + dashboard history
```

3. Technology Stack

The table below lists the technologies actually present in the repository (verified from requirements.txt and the installed virtual environment).

Technology	Version	Purpose	Where Used
Python	3.14.3 (venv); README requires 3.10+	Backend programming language	backend/
FastAPI	0.141.1	REST API framework + page/static serving	backend/main.py, backend/routes/
Uvicorn	0.52.4	ASGI server that runs the app	start.bat, run command
SQLAlchemy	2.0.52	ORM + engine/session for database access	backend/models.py, backend/database.py
Pydantic / pydantic-settings	2.13.4 / 2.15.0	Request/response validation and environment config	backend/schemas.py, backend/config.py
httpx	0.28.1	HTTP client used to call the Gemini API	backend/services/gemini_service.py
python-jose	3.5.0	JWT creation/decoding for authentication	backend/utils/security.py
bcrypt	5.0.0	Password hashing	backend/utils/security.py
python-dotenv	1.2.3	Loads .env variables	transitively via config settings
PyMySQL	1.2.0	MySQL driver (attempted first; falls back to SQLite)	backend/database.py
python-multipart	0.0.32	Form/multipart support (FastAPI dependency)	backend
SQLite	in active use (interview.db)	Development database	project root /interview.db
MySQL 8.0+	intended production DB	Production database (schema provided)	database/schema.sql
HTML5 + CSS3	-	Page structure and design system	frontend/*.html, frontend/css/style.css
Vanilla JavaScript	-	Client-side logic, fetch API calls, rendering	frontend/js/*.js

AI provider: Google Gemini REST API (not OpenAI). The README describes an OpenAI-compatible mock mode, but the running code calls the Gemini endpoint directly with httpx. This difference is documented as a known issue in Section 24.

4. Project Directory

Important directory and file layout of the repository (venv omitted):

```
ai-interview-trainer/
|-- backend/
|   |-- main.py                 FastAPI entry point, CORS, static pages, migrations
|   |-- config.py               Pydantic Settings (environment variables)
|   |-- database.py             SQLAlchemy engine + session factory
|   |-- models.py               ORM models: User, Interview, Question, Answer, Feedback
|   |-- schemas.py              Pydantic request/response schemas
|   |-- routes/
|   |   |-- auth.py             Register, login, current-user endpoints
|   |   |-- interview.py        Interview CRUD, questions, answers, complete
|   |   `-- dashboard.py        Dashboard statistics
|   |-- services/
|   |   |-- ai_service.py       Facade that delegates to gemini_service
|   |   |-- gemini_service.py   Gemini HTTP client, retries, JSON parsing, evaluation
|   |   `-- fallback_questions.py  Local MCQ/writing question banks
|   `-- utils/
|       `-- security.py         bcrypt hashing + JWT + current-user dependency
|-- frontend/
|   |-- index.html              Landing page
|   |-- login.html / register.html
|   |-- dashboard.html          Stats + history
|   |-- interview.html          Setup + live interview (MCQ/writing)
|   |-- result.html             Post-interview results
|   |-- css/style.css           Design system
|   `-- js/  api.js, auth.js, dashboard.js, interview.js, result.js
|-- database/schema.sql         MySQL reference schema
|-- .env.example                Environment variable template
|-- .env                        Actual secrets (DO NOT commit or share)
|-- requirements.txt            Python dependencies
|-- start.bat                   Windows launcher for uvicorn
|-- README.md                   Project documentation
|-- interview.db                Active SQLite database file
`-- server_out.log / server_err.log   Runtime log files
```

File roles (backend):

File	Purpose	Important functions/classes	Dependencies	Role in application
backend/main.py	Application entry point	create_app pieces: add_column_if_missing(), global_exception_handler; page routes serve_index/serve_login/serve_dashboard/serve_interview/serve_result; CORS middleware setup	FastAPI, CORS, StaticFiles, config, database, routes	Boots FastAPI, creates tables, registers routers, serves frontend
backend/config.py	Environment configuration	Settings class, get_settings() (cached)	pydantic-settings	Provides DATABASE_URL, MYSQL_*, GEMINI_API_KEY, GEMINI_MODEL, SECRET_KEY, ACCESS_TOKEN_EXPIRE_MINUTES, ALGORITHM
backend/database.py	Database engine/session	engine, SessionLocal, Base, get_db()	SQLAlchemy, PyMySQL	Connects MySQL or falls back to SQLite; every request gets a session
backend/models.py	ORM models	User, Interview, Question, Answer, Feedback; enums InterviewType/DifficultyLevel/InterviewStatus	SQLAlchemy	Defines tables and relationships
backend/schemas.py	Validation	UserCreate, UserLogin, UserResponse, Token, InterviewCreate, QuestionResponse, AnswerSubmit, AnswerResponse, FeedbackResponse, InterviewDetail, DashboardStats	Pydantic	Validates input and shapes output
backend/routes/auth.py	Authentication API	register(), login(), get_me()	SQLAlchemy, security utils	User registration and JWT login
backend/routes/interview.py	Interview API	create_interview(), get_next_question(), _determine_question_type(), _option_letter(), submit_answer(), complete_interview(), get_interview(), list_interviews()	SQLAlchemy, ai_service, schemas	Core interview logic and scoring
backend/routes/dashboard.py	Dashboard API	get_dashboard_stats()	SQLAlchemy, schemas	Aggregates interview statistics
backend/services/ai_service.py	AI facade	generate_question(), evaluate_answer(), generate_overall_feedback()	gemini_service	Unified AI entry point for routes

File	Purpose	Important functions/classes	Dependencies	Role in application
backend/services/gemini_service.py	Gemini integration	get_fallback_question import, _call_gemini(), parse_gemini_json(), _extract_mcq_from_text(), _normalize_mcq(), _generate_mcq(), generate_question(), evaluate_answer(), generate_overall_feedback()	httpx, config, fallback_questions	Calls Gemini, retries on rate limits, parses JSON, falls back to question bank
backend/services/fallback_questions.py	Offline question banks	MCQ_BANK, WRITING_BANK, get_fallback_question()	stdlib random	Provides questions when Gemini is unavailable
backend/utils/security.py	Security helpers	hash_password(), verify_password(), create_access_token(), get_current_user()	python-jose, bcrypt, SQLAlchemy	Password hashing, JWT tokens, auth dependency

File roles (frontend):

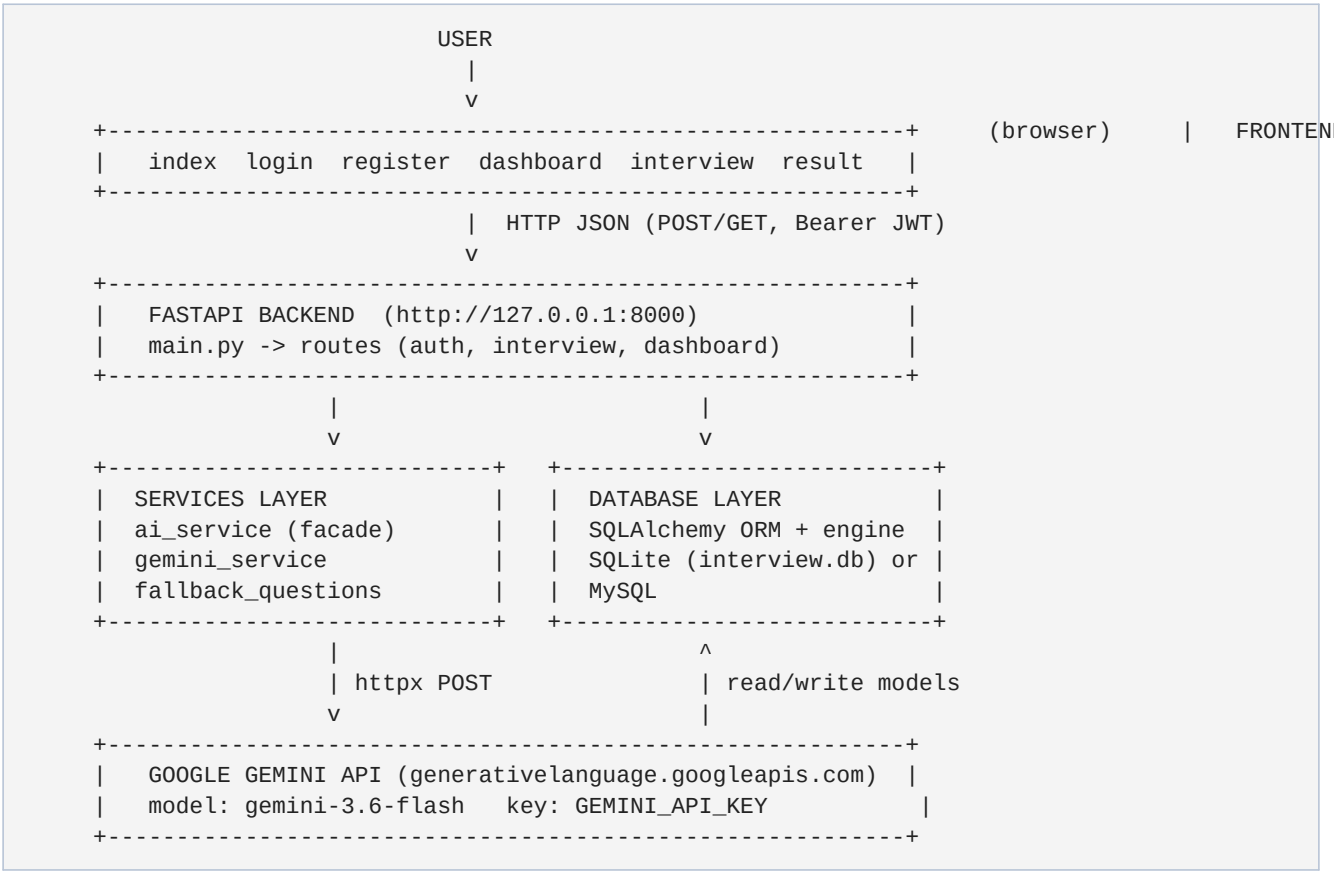
File	Purpose	Important functions	Dependencies	Role in application
frontend/js/api.js	HTTP client + token management	apiRequest(), getToken()/setToken()/clearToken(), getErrorMessage(), errorMessageToString(); api.auth, api.dashboard, api.interviews	fetch()	All backend calls; automatically attaches JWT; normalizes errors
frontend/js/auth.js	Auth forms	initAuthPage(), handleLogin(), handleRegister(), showError()	api.js	Login/register page logic
frontend/js/dashboard.js	Dashboard rendering	initDashboard(), updateNavbar(), loadDashboardStats(), renderStats(), renderInterviewHistory(), handleLogout()	api.js	Stats cards and history table
frontend/js/interview.js	Interview orchestration	initInterview(), startInterview(), loadNextQuestion(), renderMcqOptions(), submitAnswer(), showFeedback(), nextQuestion(), finishInterview()	api.js	Runs the whole interview flow in the browser
frontend/js/result.js	Results rendering	initResult(), renderResults(), truncate()	api.js	Displays overall + per-question results
frontend/*.html	Pages	index, login, register, dashboard, interview, result	style.css + js files	User interface
frontend/css/style.css	Design system	Cards, navbar, option grid, MCQ options, feedback panel, progress bar, badges, spinners	-	All styling

Database + configuration + support files:

File	Purpose	Important parts	Dependencies	Role
database/schema.sql	MySQL reference schema	CREATE DATABASE ai_interview; users, interviews, questions, answers, feedback tables with FKs	MySQL	Production DB blueprint
interview.db	Active SQLite database	Tables: users, interviews, questions, answers, feedback	SQLite	Actual dev database (5 users, 40 interviews, 41 questions, 24 answers, 24 feedback rows at inspection time)
.env.example	Environment template	DATABASE_URL, MYSQL_*, GEMINI_API_KEY, GEMINI_MODEL, SECRET_KEY	-	Shows which variables to set
.env	Actual secrets	Same names as .env.example with real values	-	Loaded at startup; do NOT commit
requirements.txt	Python dependencies	fastapi, uvicorn[standard], sqlalchemy, pymysql, python-dotenv, pydantic, pydantic-settings, python-jose[cryptography], python-multipart, httpx, bcrypt	-	pip install target
start.bat	Windows launcher	cd to script dir, checks venv python, runs uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000	venv	One-click server start
README.md	Project documentation	Features, install steps, endpoints table, future work	-	Human-readable overview (partly outdated - see Section 24)

5. System Architecture

The application follows a classic 3-tier web architecture: a browser frontend, a Python API backend, and a relational database. The AI service is an external dependency called by the backend.



5.1 How the parts communicate

- **Browser to backend:** JavaScript uses fetch() to call REST endpoints with JSON bodies. On 401 the client clears the token.
- **Auth:** The login endpoint returns a JWT which the client stores in localStorage and sends as 'Authorization: Bearer ' on every request.
- **Backend to AI:** gemini_service.py sends the Gemini API an HTTP POST with the model, a generated prompt, and generation settings. Responses are parsed as JSON when structured output is requested.
- **Backend to database:** routes use SQLAlchemy sessions to insert/query User, Interview, Question, Answer, Feedback rows.
- **Backend to frontend:** responses are serialized to JSON by FastAPI/Pydantic, and the frontend renders them into the DOM.

5.2 Why this design

Separating frontend, backend, services, and database keeps each concern independent: the UI never touches the AI API or the database directly; the AI service is isolated so it can be swapped or given a fallback; and the database layer can move from SQLite to MySQL without changing the routes. This makes the project easy to extend (timers, admin panel, voice interviews are planned future work).

6. Backend Architecture

6.1 Application entry point

backend/main.py creates the FastAPI app (title 'AI Interview Training System', version 1.0.0). On startup it calls Base.metadata.create_all() to create tables, runs idempotent ALTER TABLE migrations that add question_type, options and correct_answer to the questions table, registers the CORS middleware, installs the three routers (auth, interview, dashboard), mounts the frontend as static files, defines the HTML page routes, and adds a global exception handler that returns a JSON 500 response.

6.2 Configuration

backend/config.py defines a pydantic-settings Settings class that reads a .env file. Relevant variables include DATABASE_URL, MYSQL_HOST/PORT/USER/PASSWORD/DATABASE, GEMINI_API_KEY, GEMINI_MODEL, SECRET_KEY, ACCESS_TOKEN_EXPIRE_MINUTES (default 60) and ALGORITHM (default HS256). get_settings() is cached with lru_cache so settings load once.

6.3 Database access

backend/database.py tries to connect to MySQL through PyMySQL. If that fails it creates a SQLite engine pointing at interview.db in the project root and prints 'MySQL not available. Using SQLite'. SessionLocal creates sessions; get_db() is a FastAPI dependency that yields a session and closes it afterwards.

6.4 Authentication

backend/utils/security.py hashes passwords with bcrypt (hash_password/verify_password), creates JWTs with python-jose (create_access_token), and provides get_current_user(), a FastAPI dependency that decodes the Bearer token, extracts the user id from the 'sub' claim, and loads the User from the database. Failing tokens raise HTTP 401.

6.5 Routes, services, schemas, models

- **routes/** - FastAPI routers that receive HTTP requests, validate with Pydantic schemas, use the model layer, and return responses.
- **services/** - ai_service is a facade; gemini_service contains all Gemini calls and JSON handling; fallback_questions supplies offline question banks.
- **schemas/** - request bodies (UserCreate, UserLogin, InterviewCreate, AnswerSubmit) and response models (UserResponse, Token, QuestionResponse, AnswerResponse, FeedbackResponse, InterviewResponse, InterviewDetail, DashboardStats).
- **utils/security.py** - auth helpers used by routes.
- **models/** - ORM classes described in Section 8.

6.6 Error handling, logging, CORS

- CORS: allowed origins from CORS_ORIGINS env var (default http://localhost:8000, http://127.0.0.1:8000).
- Global exception handler in main.py returns {'detail': 'An internal server error occurred...'} with HTTP 500.
- Logging: a logger named 'ai_interview' at INFO level; uvicorn writes server_out.log / server_err.log when started with output redirection.

- Endpoints raise HTTPException with readable detail strings (404, 400, 503, 401) which the frontend displays.

6.7 API table

Method	Endpoint	Purpose	Request body	Response	File
POST	/api/auth/register	Create user account	UserCreate (full_name, email, password)	201 UserResponse	routes/auth.py
POST	/api/auth/login	Authenticate + issue JWT	UserLogin (email, password)	Token (access_token, token_type)	routes/auth.py
GET	/api/auth/me	Current user profile	- (Bearer)	UserResponse	routes/auth.py
GET	/api/dashboard/stats	Dashboard statistics	- (Bearer)	DashboardStats	routes/dashboard.py
POST	/api/interviews	Create interview	InterviewCreate (job_role, interview_type, difficulty, total_questions)	201 InterviewResponse	routes/interview.py
GET	/api/interviews	List user interviews	- (Bearer)	list[InterviewResponse]	routes/interview.py
GET	/api/interviews/{id}	Interview detail with Q&A;	- (Bearer)	InterviewDetail	routes/interview.py
POST	/api/interviews/{id}/questions	Get next AI-generated question	- (Bearer)	QuestionResponse (question_text, question_type, options)	routes/interview.py
POST	/api/interviews/{id}/answers	Submit answer + evaluate	AnswerSubmit (answer_text and/or selected_option)	AnswerResponse (score, feedback)	routes/interview.py
POST	/api/interviews/{id}/complete	Finish interview + overall feedback	- (Bearer)	InterviewDetail	routes/interview.py

6.8 HTML page routes served by the backend

```
GET /            -> frontend/index.html
GET /login       -> frontend/login.html
GET /register    -> frontend/register.html
GET /dashboard   -> frontend/dashboard.html
GET /training    -> frontend/interview.html
GET /interview   -> frontend/interview.html
GET /result      -> frontend/result.html
GET /static/*    -> files under frontend/  (css, js)
GET /docs        -> auto-generated FastAPI Swagger UI
```

7. Frontend Architecture

The frontend is built with plain HTML5, CSS3 and Vanilla JavaScript - no framework. Every page loads js/api.js first (the shared HTTP client), then a page-specific script. The backend serves the HTML directly, so there is no separate frontend server.

7.1 Pages

Page	URL	Purpose	Key JS
index.html	/	Landing page describing the product	inline script (login check)
login.html	/login	Sign-in form	auth.js
register.html	/register	Account creation form	auth.js
dashboard.html	/dashboard	Stats cards + interview history table	dashboard.js
interview.html	/interview	Interview setup + live question/answer/feedback UI	interview.js
result.html	/result?id=N	Overall score + per-question feedback	result.js

7.2 Shared HTTP client (api.js)

- Stores the JWT in localStorage (token key), attaches it as a Bearer header, and clears it on HTTP 401.
- apiRequest() parses JSON responses, turns FastAPI error bodies into readable messages using getErrorMessage()/errorMessageToString() (prevents '[object Object]' display).
- Exposes namespaced helpers: api.auth (register/login/getMe/logout), api.dashboard.getStats, api.interviews (create/getQuestion/submitAnswer/complete/getOne/getAll).

7.3 Interview UI (interview.js)

- **Setup view:** option cards for job role (6 presets + custom 'Other'), interview type, difficulty, and question count (5/10). Hidden inputs hold the selected values.
- **Loading states:** 'Generating question...' spinner the whole time the backend asks the AI; loading overlay with a spinner-large element.
- **MCQ view:** renderMcqOptions() builds radio labels from currentQuestion.options; getSelectedMcqOption() returns the checked value.
- **Writing view:** a textarea (answerText); validation requires non-empty text.
- **Submit flow:** disables controls, shows 'Evaluating...', calls submitAnswer, then showFeedback() renders score badge (high/medium/low class), strengths list, weaknesses list and suggestions list; hides Submit and reveals Next Question.
- **Error states:** setup errors, answer errors and question-load errors displayed in alert-error divs via getErrorMessage().
- **Progress sidebar:** progress bar fill, question count, role, difficulty and type labels.
- **Finish:** calls complete endpoint then redirects to /result?id=...

7.4 Results UI (result.js)

result.js fetches the interview detail, renders the average score circle (excellent >= 8, good >= 6, average >= 4, else poor), the overall feedback paragraph, and one card per answered question with score, candidate answer (truncated), strengths, weaknesses and a collapsible model answer.

7.5 Loading, error and empty states

- Buttons show spinner markup while requests are in-flight.
- Answer/error alerts are hidden by default and shown only when needed.
- Dashboard history table shows an empty-state row when the user has no interviews.

7.6 Admin UI

No admin UI exists. There are no admin pages, no admin routes, and no role field on the User model. (See Section 15.)

8. Database

8.1 Database type and file

In use: SQLite file interview.db in the project root (because MySQL is not available on the developer machine). **Intended production database:** MySQL 8.0+, defined in database/schema.sql. SQLAlchemy is the ORM used for both. The engine and session are created in backend/database.py.

8.2 Actual tables (verified from interview.db)

Table	Rows (at inspection)	Important columns
users	5	id (PK), full_name, email (unique), hashed_password, created_at
interviews	40	id (PK), user_id (FK -> users.id), job_role, interview_type, difficulty, total_questions, current_question, total_score, status, overall_feedback, created_at, completed_at
questions	41	id (PK), interview_id (FK -> interviews.id), question_number, question_text, question_type, options (JSON text), correct_answer, created_at
answers	24	id (PK), question_id (FK -> questions.id), candidate_answer, score, created_at
feedback	24	id (PK), answer_id (FK -> answers.id), strengths, weaknesses, suggestions, model_answer, created_at

8.3 Relationship diagram (matches ORM models)

```
users(id)              has many
   | 1
   +---- interviews(id, user_id)     has many
           | 1
           +---- questions(id, interview_id, question_type,
                           options, correct_answer)
                  | 1
                  +---- answers(id, question_id, candidate_answer, score)
                          | 1
                          +---- feedback(id, answer_id,
                                         strengths, weaknesses,
                                         suggestions, model_answer)

User -> interviews -> questions -> answers -> feedback   (one-to-many chain)
Each question has at most one answer (uselist=False).
```

8.4 Important queries / CRUD

- **Create:** db.add(object) + db.commit() (register user, create interview, save question, save answer, save feedback).
- **Read:** db.query(Model).filter(...).first() or .all(); interview detail uses joinedload to fetch questions, answers and feedback in one query.
- **Update:** interview.current_question += 1; interview.total_score += score; interview.status='completed'; commit.

- **Delete:** none implemented in routes (cascade rules exist in the model: deleting a User removes interviews/questions/answers/feedback).
- **Aggregation:** dashboard averages completed interviews; complete_interview averages question scores.

9. AI Integration

9.1 Provider, model, environment variables

Item	Value
AI provider	Google Gemini (generative language API)
SDK/library	None - raw HTTP via httpx (no Google SDK installed)
Model	Read from GEMINI_MODEL (default 'gemini-3.6-flash')
Endpoint	https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent
API key env var	GEMINI_API_KEY (value REDACTED in this document)
JWT secret env var	SECRET_KEY (value REDACTED)

9.2 Client initialization

There is no client object. gemini_service.py builds the request per call: it validates the key with _get_gemini_key() (raises ValueError if the key is missing or still the placeholder 'YOUR_NEW_GEMINI_API_KEY'), formats the URL with the model, and sends an HTTP POST with params key=api_key and a JSON payload with contents[]/parts[]/text plus generationConfig (temperature, maxOutputTokens).

9.3 Prompt generation

- **MCQ question:** role, interview type, difficulty, question number, previously asked questions, rules (4 short distinct options, one correct answer, under 40 words), and a strict JSON-only instruction. A first prompt and a stricter one-line prompt are attempted in order.
- **Written question:** role/type/difficulty/number plus instructions to return question text only.
- **Answer evaluation:** the question, candidate answer, role/type/difficulty, a scoring guide (1-10) and a request for a JSON object with score, strengths, weaknesses, suggestions, model_answer.
- **Overall feedback:** role, difficulty, average score, per-question scores and a request for a 3-5 sentence feedback paragraph.

9.4 Request and retry/rate-limit handling

_call_gemini() retries up to MAX_RETRIES=3 attempts with exponential backoff (_backoff_delay: 2s then doubled, capped at 30s; honors the Retry-After header capped at 60s). It handles HTTP 400 (bad request message), 403 (invalid key), 429 (rate limit), timeouts (httpx.TimeoutException), connection errors, empty candidates/parts, and unexpected exceptions. After 3 failed attempts it raises a RuntimeError with a user-friendly message. Timeout per request is 60 seconds.

9.5 Response handling and JSON parsing

- Successful responses return the text of candidates[0].content.parts[0].text.
- parse_gemini_json() tries to load pure JSON, then JSON inside code fences, then any {...} block in the text, returning None if all fail.

- _extract_mcq_from_text() is a regex fallback for MCQs that are not valid JSON (finds the question, options on lines like 'A. text', and the correct-answer letter).
- _normalize_mcq() validates exactly 4 non-empty options, maps the correct answer to the letter A-D, strips leading letters from option text, and produces lettered options (for example 'A. O(n)') plus the correct letter.
- evaluate_answer() verifies all required keys exist (score, strengths, weaknesses, suggestions, model_answer), clamps score to 1-10, and coerces string fields to lists.

9.6 Fallback behavior

If MCQ generation fails on both prompt attempts (e.g., rate limit), _generate_mcq() returns a question from the local bank using get_fallback_question(). Written questions do the same on RuntimeError or empty output. This guarantees the interview never gets stuck when Gemini is unavailable. The banks are in backend/services/fallback_questions.py (generic and Software Developer entries for MCQ and writing).

9.7 AI flow diagram

```
USER
  |  submits answer / requests question
  v
BACKEND  (routes/interview.py)
  |  calls services/ai_service.py (facade)
  v
AI SERVICE  (services/gemini_service.py)
  |  builds prompt text
  v
PROMPT  (role, difficulty, question, answer, rules)
  |  httpx.post (key + model + generationConfig)
  v
GEMINI API (server-side AI inference)
  |  returns text (JSON for MCQ/evaluation)
  v
VALIDATION + PARSING  (parse_gemini_json / _normalize_mcq,
                       required keys check in evaluate_answer)
  |  on failure retry -> fallback question bank
  v
APPLICATION  (stores question/answer/feedback, computes score)
  |  serialized by Pydantic
  v
FRONTEND  (renders question/score/feedback)
```

10. Interview Flow

Implemented steps, in order:

```
1.  User opens the application (GET /)
2.  User registers or logs in (POST /api/auth/register or /login);
    browser stores the JWT token
3.  User lands on /dashboard after login (or /training directly)
4.  User presses Start Interview -> /interview
5.  User selects job role, interview type, difficulty, question count
6.  Start -> POST /api/interviews creates the interview (in_progress)
7.  POST /api/interviews/{id}/questions generates and stores the next
    question (AI or fallback bank), increments current_question
8.  Question is displayed (MCQ radio options or writing textarea)
9.  User answers:
      MCQ         -> selected_option = full option string (e.g. 'B. O(log n)')
      Writing     -> answer_text
10. POST /api/interviews/{id}/answers evaluates and stores the result
    - MCQ: letter match -> score 10 or 0
    - Writing: Gemini returns score 1-10 + feedback
11. Feedback panel shows score, strengths, weaknesses, suggestions
12. Next Question -> repeat from step 7 until questionCount reaches
    total_questions
13. Last question -> 'Finish Interview' -> POST /api/interviews/{id}/complete
    - generates overall feedback, sets status=completed,
      total_score = average of question scores
14. Redirect to /result?id=... which renders the summary
15. Results are persisted; dashboard lists the interview history
```

11. Question System

11.1 Generation and selection

Question type is decided by the backend (routes/interview.py, _determine_question_type) from the difficulty and question number - the frontend does not choose question types. generate_question() in gemini_service.py builds a role/difficulty-specific prompt and asks Gemini for one question. If the AI call fails, a question is taken from the local fallback bank (MCQ_BANK / WRITING_BANK) so the interview still progresses.

11.2 Roles, interview types, difficulty, count

- Job roles offered in the UI: Software Developer, Python Developer, Java Developer, Web Developer, Data Analyst, or a custom role typed by the user.
- Interview types (validated by schema pattern): technical, hr_behavioral, mixed.
- Difficulty (validated by schema pattern): beginner, intermediate, advanced.
- Number of questions: UI offers 5 or 10; schema allows any value 5-20 (ge=5, le=20).

11.3 Storage

Each generated question is stored as a Question row with question_number, question_text, question_type ('mcq' or 'writing'), options (JSON string of lettered options for MCQs, otherwise NULL) and correct_answer (the A-D letter for MCQs, otherwise NULL). The columns question_type, options and correct_answer are added by an idempotent migration in main.py.

11.4 MCQ rules and options

- The AI is told to return exactly 4 short, distinct, mutually exclusive options with exactly one correct answer and a concise question.
- _normalize_mcq() enforces 4 non-empty options and maps correct_answer to a letter.
- Options are stored letter-prefixed ('A. ') so the frontend can render them as radio labels and the backend can recover the letter via _option_letter().

11.5 Next-question logic

loadNextQuestion() in the frontend stops when questionCount reaches total_questions and instead calls finishInterview(). The backend rejects question requests when current_question >= total_questions or when the interview is not in_progress. The backend also avoids repeating answers: submitting to an already-answered question returns 'Already answered this question'.

12. Difficulty System

Verified implementation in backend/routes/interview.py, function _determine_question_type():

```
def _determine_question_type(difficulty, question_number):
    if difficulty == 'beginner':        return 'mcq'      # EASY  = 100% MCQ
    if difficulty == 'advanced':        return 'writing'  # HARD  = 100% writing
    if question_number % 2 == 1:        return 'mcq'      # MEDIUM odd  = MCQ
    return 'writing'                                      # MEDIUM even = writing
```

Requirement	Actual behavior	Correct?
EASY: 100% MCQ	beginner -> every question_type is 'mcq'	Yes
MEDIUM: approx. 50% MCQ / 50% writing	intermediate alternates MCQ (odd numbers) and writing (even numbers). For 5 questions that is 3 MCQ + 2 writing (ratio 60/40); for 10 questions 5/5 (50/50).	Approximately yes (not exactly 50% for odd totals)
HARD: 100% writing	advanced -> every question_type is 'writing'	Yes
Randomness used?	No - a deterministic alternation based on question_number % 2.	Documented behavior
Frontend/backend agree?	Frontend does not decide types; it renders whatever question_type the backend returns and shows radios for 'mcq' and a textarea otherwise.	Yes

Note: the difficulty value is also passed into the AI prompts so question content can match the level, but the MCQ-vs-writing decision is purely the deterministic function above. No timer penalties are applied regardless of difficulty (see Section 13).

13. Timer System

No timer is implemented. There is no per-question countdown, no timeout submission, and no mark reduction for slow answers anywhere in the codebase.

```
Expected requirement              Actual implementation
--------------------------------  ------------------------------------
2-minute timer per question       NOT IMPLEMENTED (no countdown logic)
Reduced marks when time expires   NOT IMPLEMENTED
Automatic submission on timeout   NOT IMPLEMENTED
Timer reset for next question     NOT IMPLEMENTED
Timeout storage                   NOT IMPLEMENTED (no timer column/tables)
```

Evidence: no setTimeout/interval countdown in the JS, no timer fields in the ORM models or schema, and the README lists 'Per-question timer' under 'Future Enhancements'. The user is free to answer at any pace. If a per-question timer is required, it must be added to the frontend (display + auto-submit), the backend scoring (optional mark reduction), and the database (optional timeout storage).

14. Scoring System

14.1 Per-question scoring

Question type	Score logic	Where	Values
MCQ	Selected letter compared with stored correct letter; equal -> 10, otherwise 0	routes/interview.py submit_answer()	0 or 10
Written	Gemini returns a JSON 'score' field; clamped to 1-10 by evaluate_answer()	services/gemini_service.py evaluate_answer()	1-10 (AI judged)

14.2 Interview totals

- As questions are answered, interview.total_score accumulates the per-question scores.
- On completion, complete_interview() replaces total_score with the average: round(sum(scores) / len(scores), 1).
- Dashboard computes average and highest score only over completed interviews.

14.3 Result percentage and grading

The results page derives a percentage = score/10 * 100 for the score circle and labels the average as Excellent (>= 8), Good (>= 6), Average (>= 4) or Needs Improvement (< 4). The feedback panel colors a score badge high/medium/low with thresholds 7 and 5.

14.4 Strengths / weaknesses / suggestions

For MCQs, feedback is generated deterministically ('Correct answer' / 'Incorrect option selected' plus 'The correct answer is: X'). For written answers, Gemini produces strengths, weaknesses, suggestions and a model_answer, which are validated and normalized by schema validators (string-to-list handling in FeedbackResponse).

14.5 Timer penalties

None - scoring is unaffected by time because no timer exists (Section 13).

15. Admin System

ADMIN PANEL NOT IMPLEMENTED.

- There is no admin login, no role column on the User model, and no admin routes or pages.
- No endpoints exist to manage users, interviews, questions, answers, scores or statistics beyond the logged-in user's own data.
- The README lists 'Admin panel' under Future Enhancements.

Consequence: every user only ever sees and modifies their own interviews (all routes filter by Interview.user_id == current_user.id).

16. Authentication & Security

16.1 Registration and login

- Registration: POST /api/auth/register validates with Pydantic (name 2-100 chars, password 6-100), rejects duplicate emails (400 'Email already registered'), hashes the password with bcrypt, and stores hashed_password.
- Login: POST /api/auth/login checks the email and verify_password(); success returns a JWT (access_token, token_type='bearer') valid for ACCESS_TOKEN_EXPIRE_MINUTES (default 60).
- JWT: python-jose signs the token with SECRET_KEY using HS256; 'sub' is the user id.
- Authorization: protected endpoints depend on get_current_user(), which decodes the token and loads the user; invalid/missing tokens produce 401.

16.2 Password safety

Passwords are never stored as plain text. bcrypt generates a salted hash via hash_password(); only the hash is stored. verify_password() compares in constant time using bcrypt.checkpw.

16.3 Frontend token handling

The token is saved to localStorage, attached as Authorization: Bearer , and cleared on a 401 response. The login page watches for a missing token and redirects unauthenticated users to /login.

16.4 Other measures and notes

```
API-key protection   GEMINI_API_KEY and SECRET_KEY live in .env, read by
                     backend/config.py; they are never exposed to the
                     frontend. .gitignore excludes .env.
SQL injection        Queries use SQLAlchemy ORM / parametrized statement
                     objects, not string concatenation.
Input validation     Pydantic schemas enforce types, lengths, patterns
                     (role pattern, difficulty pattern, total_questions 5-20).
CORS                 Only localhost origins allowed by default.
HTTP                 Development runs on http (no TLS). HTTPS is
                     recommended for real deployments.
Database security    SQLite file has no credentials; MySQL would use the
                     MYSQL_* variables. Schema.sql uses utf8mb4.
Sensitive data       .env must never be committed/shared. This document
                     redacts all secret values.
```

16.5 Security concerns to note

- localStorage is readable by any script on the same origin (acceptable for a demo; HttpOnly cookies are more secure).
- SECRET_KEY has a weak default ('change-this-to-a-random-secret-string') if .env is not configured; production must set a strong random value.
- No HTTPS in development; fine for local practice, insufficient for real users without TLS termination.
- No email verification or password reset (listed as future work).
- Global exception handler returns a generic 500 to avoid leaking internals (logs full error server-side).

17. Error Handling

Known / previously observed error scenarios and how the application handles them:

Error	Source file (function)	Root cause	Impact	Current handling	Recommended fix	Priority
Failed to load the question. Please try again.	frontend/js/interview.js (loadNextQuestion catch)	Backend returned 503 (Gemini down/rate-limited) or another question-generation failure	User cannot continue the interview	Frontend shows the error and a Retry button wired to retryLoadQuestion(); backend already falls back to the local question bank	Generally mitigated by the fallback bank; keep Retry as a safety net	High
AI returned an invalid MCQ question.	backend/services/gemini_service.py (_normalize_mcq)	Older Gemini responses without valid question/options/correct_answer	Question generation failed	_normalize_mcq raises ValueError; _generate_mcq retries the stricter prompt, then uses the fallback question bank	Covered by current design (fallback bank)	Med
AI returned invalid JSON for MCQ.	gemini_service.py (parse_gemini_json / _extract_mcq_from_text)	Gemini emits prose, markdown fences or escaped quotes instead of clean JSON	Structured parsing failed	parse_gemini_json tries multiple strategies; _extract_mcq_from_text regex-parses letters/options; fallback bank as last resort	Covered; consider logging the raw text for debugging	Med
AI service rate limit exceeded. Please try again later.	gemini_service.py (_call_gemini)	Gemini HTTP 429 (quota exhausted)	AI question/evaluation temporarily unavailable	Exponential backoff (MAX_RETRIES=3, Retry-After honored); question generation falls back to bank; evaluation raises 503	Monitor quota; optionally queue/retry written evaluations with a fallback generic feedback	High

Error	Source file (function)	Root cause	Impact	Current handling	Recommended fix	Priority
[object Object]	frontend/js/api.js (apiRequest) + frontend/js/interview.js (error display)	FastAPI validation errors (422) returned detail as an array of objects; the old code did new Error(data.detail), so error.message became '[object Object]' which was rendered into alert divs	Confusing red-box message after answer submission	Fixed: errorMessageToString()/getErrorMessage() convert detail arrays/objects to readable strings; backend also widened AnswerSubmit.selected_option max_length 10 -> 500 so valid full option strings are accepted	Also switchthe writing-answer catch in interview.js (line ~317) from error.message to getErrorMessage() for full consistency	High

17.1 Server-side error surfacing

Routes raise HTTPException with short user-friendly details (404/400/503/401). The global exception handler converts unexpected exceptions into a generic JSON 500. Gemini-specific messages say clearly whether the key is missing, invalid, rate-limited or unreachable.

18. Logging & Debugging

18.1 Where logs live

Log source	Location	What it shows
Uvicorn stdout/stderr	server_out.log / server_err.log when the server is started with output redirection	Startup messages, request lines, and all python logging output
App logger 'ai_interview'	Server output (console or the log files)	INFO/WARNING/ERROR: question requests, generation success/failure, Gemini retries, validation failures, evaluation errors
Gemini HTTP layer	Server output	Every Gemini request including status codes (e.g. HTTP 429 Too Many Requests)
Browser console	DevTools (F12) console	Frontend errors, 'Non-JSON response' messages, failed dashboard loads
Frontend alerts	Alerts on the page	User-facing error messages from getErrorMessage()
Database	No dedicated DB log; inspect interview.db tables	users, interviews, questions, answers, feedback (for state debugging)

18.2 Debugging guide

- Question does not load: check server_err.log for 'Gemini request attempt' / '429' / 'fell back to the local question bank' / 'Unhandled exception'.
- Login fails: check POST /api/auth/login responses (401), verify the user exists in the users table, and check SECRET_KEY consistency across restarts.
- Wrong score or feedback: read the answers/feedback rows for the interview; confirm MCQ correct_answer letter vs selected option.
- Server unreachable: verify port 8000 listening (Get-NetTCPConnection -LocalPort 8000) and that uvicorn is still running.
- [object Object]: this specific bug is fixed in frontend/js/api.js; if it reappears, check that the page loads the updated api.js (browser cache) and that errorMessageToString/getErrorMessage handle the shape received.

19. Configuration

19.1 Environment variables

Loaded from .env by backend/config.py. Values are REDACTED here; only names and defaults are shown.

Variable	Purpose	Default / example
DATABASE_URL	SQLAlchemy connection string	mysql+pymysql://root:password@localhost:3306/ai_interview (falls back to SQLite interview.db)
MYSQL_HOST / MYSQL_PORT	MySQL host and port	localhost / 3306
MYSQL_USER / MYSQL_PASSWORD	MySQL credentials	root / [REDACTED]
MYSQL_DATABASE	MySQL database name	ai_interview
GEMINI_API_KEY	Google Gemini API key	[REDACTED]
GEMINI_MODEL	Gemini model id	gemini-3.6-flash
SECRET_KEY	JWT signing secret	[REDACTED] (default: change-this-to-a-random-secret-string)
ACCESS_TOKEN_EXPIRE_MINUTES	JWT lifetime	60
ALGORITHM	JWT algorithm	HS256
CORS_ORIGINS	Comma-separated allowed origins	http://localhost:8000,http://127.0.0.1:8000 (read via os.environ in main.py)

19.2 Host / port / development settings

- Server: uvicorn backend.main:app on 127.0.0.1:8000 (start.bat uses --reload; README documents --host 0.0.0.0 as an option).
- Frontend is served by the same FastAPI process (no separate static server).
- development: SQLite fallback, live reload, verbose INFO logging, CORS open only to localhost.
- production (not set up): would use MySQL, a strong SECRET_KEY, a real GEMINI_API_KEY, HTTPS termination, and locked-down CORS.

20. Dependencies

Declared in requirements.txt; versions verified from the installed virtual environment. Transitive packages used by the framework are shown as supporting packages.

Package	Version	Purpose	Used by
fastapi	0.141.1	Web framework / REST API + static serving	backend/main.py, routes
uvicorn[standard]	0.52.4	ASGI server	start.bat, run command
sqlalchemy	2.0.52	ORM + engine/session	backend/models.py, database.py, routes
pymysql	1.2.0	MySQL driver	backend/database.py
python-dotenv	1.2.3	.env loading	pydantic-settings backend
pydantic	2.13.4	Data validation	backend/schemas.py
pydantic-settings	2.15.0	Settings from environment	backend/config.py
python-jose[cryptography]	3.5.0	JWT encode/decode	backend/utils/security.py
python-multipart	0.0.32	Multipart/form support	FastAPI runtime
httpx	0.28.1	HTTP client for Gemini API	backend/services/gemini_service.py
bcrypt	5.0.0	Password hashing	backend/utils/security.py
starlette	1.6.0	FastAPI foundation (routing/middleware/static)	transitive
anyio / httpx stack	-	Async I/O + HTTP internals	transitive
python-jose deps (ecdsa, rsa, pyasn1)	-	Cryptography for JWT	transitive
reportlab	5.0.1	Used only to generate THIS documentation PDF	generation script (not a runtime dependency)

Note: reportlab was installed into the virtual environment to create this PDF and is not part of the application runtime.

21. How to Run Manually on Windows

Commands below are based on the actual repository (start.bat, README.md, requirements.txt, and the venv layout). Run them from a PowerShell or Command Prompt window.

21.1 1) Check Python

```
cd "C:\Users\sudha\OneDrive\Desktop\AI INTERVIEWER\ai-interview-trainer"python --version       (optiona
```

21.2 2) Activate the virtual environment

```
.\venv\Scripts\Activate.ps1      (PowerShell; if blocked, run: Set-ExecutionPolicy -Scope Process Bypas
```

21.3 3) Install dependencies

```
.\venv\Scripts\python.exe -m pip install -r requirements.txt
```

21.4 4) Configure environment

```
copy .env.example .env
rem Then open .env and set:  GEMINI_API_KEY=<your Gemini key>   SECRET_KEY=<random string>
rem Optionally set MYSQL_* and DATABASE_URL if MySQL should be used.
```

If GEMINI_API_KEY is missing or rate-limited, the app still works because question generation falls back to the built-in question bank; written-answer evaluation will then return 503 until the key is valid again.

21.5 5) Start the backend

```
.\venv\Scripts\python.exe -m uvicorn backend.main:app --host 127.0.0.1 --port 8000rem or use the provide
```

Equivalent documented README command (binds all interfaces, with live reload):

```
.\venv\Scripts\python.exe -m uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000
```

21.6 6) Open the browser

```
http://127.0.0.1:8000     or    http://localhost:8000API docs (Swagger UI): http://127.0.0.1:8000/docs
```

21.7 7) Stop the server

```
Ctrl + C    (in the uvicorn console window)rem PowerShell cleanup if the window is lost:$conn = Get-NetT
```

21.8 8) Check the port / 9) Restart

```
Get-NetTCPConnection -LocalPort 8000      (shows the listener PID)rem Restart = repeat step 21.5 with th
```

22. Manual Testing Guide

Plan covering the main flows. 'N/A' means the feature is not implemented (timer, admin).

Test ID	Test name	Precondition	Steps	Expected result
T01	Application startup	Server stopped	Run step 21.5; wait for start; open http://127.0.0.1:8000	Landing page loads; no error in server_err.log
T02	Login	Registered user	Open /login, enter valid credentials	Redirect to /training (or /dashboard); token stored
T03	Dashboard	Logged in	Open /dashboard	Stats cards + interview history rendered
T04	Start interview	Logged in	Open /interview, choose role/type/difficulty/count, press Start	Interview created; first question loads
T05	Easy interview	Beginner selected	Start a beginner interview	Every question is MCQ (4 options)
T06	Medium interview	Intermediate selected	Start an intermediate interview	Odd questions MCQ, even questions writing
T07	Hard interview	Advanced selected	Start an advanced interview	Every question is a writing question
T08	MCQ question	MCQ shown	Verify options render as radio labels	Exactly 4 selectable options
T09	Written question	Writing shown	Verify textarea appears	Textarea enabled; MCQ options hidden
T10	Submit answer	Question shown	Select answer (MCQ) or type text (writing), press Submit	Feedback panel appears with score, strengths, weaknesses, suggestions
T11	Timer	Any question	Look for countdown UI	N/A - timer not implemented
T12	Timeout	Any question	Wait for auto-submit	N/A - timer not implemented
T13	Score	Answer evaluated	Read feedback score	MCQ: 10 or 0. Writing: 1-10 from AI. Overall average on /result
T14	Evaluation	Answer evaluated	Read feedback panel	Readable text, never '[object Object]'
T15	Next question	Feedback shown	Press Next Question	Next question loads; counter increments
T16	Final result	Last question answered	Press Finish Interview	Redirect to /result and show overall score + feedback
T17	Admin	-	Look for admin features	N/A - admin panel not implemented
T18	AI error	Gemini unavailable	Start an interview with API down	Fallback question bank used; question still loads
T19	Rate limit	Gemini 429	Trigger multiple questions quickly	Retries/backoff then fallback bank for questions; 503 message for evaluations

Test ID	Test name	Precondition	Steps	Expected result
T20	Invalid JSON	Gemini returns prose	Request an MCQ and inspect behavior	parse_gemini_json/_extract_mcq_from_text recover or fallback bank used

23. Terminal Command Reference

Useful Windows (PowerShell) commands for this project. Path prefix examples use the project folder.

```
cd "C:\Users\sudha\OneDrive\Desktop\AI INTERVIEWER\ai-interview-trainer"# Python / environment.\venv\Scr
```

There is no git repository in this folder (no .git), so git status is not available here.

24. Current Bugs & Recommendations

Findings from the code review. No source changes were made for this document.

24.1 Critical

Bug	File	Root cause	Impact	Recommended fix
Rate-limited or invalid Gemini key can block written-answer evaluation	backend/services/gemini_service.py (_call_gemini, evaluate_answer)	HTTP 429/403 during evaluation raises 503 with no fallback for written answers	Written questions cannot be evaluated until Gemini recovers	Add a graceful fallback evaluation message or retry delay surfaced to the user; monitor quota

24.2 High

Bug	File	Root cause	Impact	Recommended fix
README documents an OpenAI-compatible/mock path that the code does not implement	README.md	Documentation drifted from implementation (code uses Google Gemini, no AI_MOCK_MODE flag)	Confuses developers and viva explanations	Update README to describe Gemini + fallback bank
Potential duplicate fallback questions in one interview	backend/services/fallback_questions.py	Bank index = question_number-1 -> modulo; 10-question interviews with a 5-entry bank repeat entries (e.g. Q6 == Q1)	Repeated MCQs in Easy interviews	Enlarge banks or add a used-questions shuffle
Writing-answer error path still uses error.message	frontend/js/interview.js (submitAnswer catch, writing branch)	Does not route through getErrorMessage()	An object-shaped error would again display as '[object Object]'	Use getErrorMessage(error) like the MCQ path

24.3 Medium

Bug	File	Root cause	Impact	Recommended fix
Medium difficulty is not exactly 50/50 for odd totals	backend/routes/interview.py (_determine_question_type)	Deterministic alternation; 5 questions -> 3 MCQ + 2 writing	Minor spec mismatch	Accept determinism or randomize the start parity
No per-question timer	whole app	Feature not implemented	Spec says timer; app has none	Implement frontend countdown + optional scoring penalty
SECRET_KEY weak default	backend/config.py	Fallback default value if .env missing	JWT forgeable if attackers know the default	Require a strong random SECRET_KEY in production

24.4 Low

Bug	File	Root cause	Impact	Recommended fix
Login loop: 401 is silently handled	frontend/js/api.js (apiRequest)	401 clears the token and returns null; some call sites assume a response	Redirects can feel abrupt	Add an explicit redirect to /login on 401
result.js uses error.message directly	frontend/js/result.js	Does not use getErrorMessage()	Possible raw object display on load failure	Use getErrorMessage()
No pagination for long interview history	backend/routes/dashboard.py, frontend/dashboard.js	History limited to 10 recent rows in stats	Very old interviews invisible from dashboard	Add pagination endpoint

25. Interview Explanation

25.1 30-second explanation

AI Interview Trainer is a web app for practising job interviews. A user logs in, picks a role, difficulty and interview type, then answers AI-generated questions. Every answer is scored: multiple-choice questions get 10 or 0, written answers are evaluated by the Google Gemini AI into a score plus strengths, weaknesses and suggestions. Results are saved and shown on a dashboard, so candidates can track their improvement over time.

25.2 2-minute explanation

The app combines a JavaScript frontend, a Python FastAPI backend and a database, with Gemini as the AI engine. Registration and login use JWT tokens; passwords are hashed with bcrypt. Once inside, a user configures an interview - role, type (technical, HR/behavioral or mixed), difficulty (beginner, intermediate, advanced) and 5 or 10 questions. The backend decides question format: beginner is always MCQ, advanced is always written, and intermediate alternates. Question text comes from Gemini and is stored in the questions table with its options and correct answer; if the AI is unavailable, a local question bank is used so the interview never stalls. When the user submits, MCQs are checked against the stored correct letter, while written answers are sent to Gemini, which returns a JSON evaluation. Each answer and its feedback are stored, scores accumulate, and completing the interview produces an average score and overall feedback on the results page. The dashboard aggregates totals and averages for statistics and history.

25.3 5-minute technical explanation

Architecturally there are three layers. The **presentation layer** is six static HTML pages and five JavaScript files. api.js centralizes HTTP and error handling; interview.js orchestrates the interview state machine (setup, loading, MCQ vs writing rendering, submit, feedback, next, finish). The **application layer** is FastAPI: routers validate input through Pydantic schemas, enforce ownership with the get_current_user dependency, and orchestrate services and the database. The **service layer** isolates the AI: ai_service is a facade, gemini_service contains the Gemini HTTP client with 3-attempt exponential backoff, JSON cleaning (parse_gemini_json, _extract_mcq_from_text) and normalization (_normalize_mcq), and fallback_questions guarantees availability. The **data layer** uses SQLAlchemy ORM: five related tables (users, interviews, questions, answers, feedback) on SQLite with a MySQL schema provided for production. Security relies on bcrypt hashing, JWT bearer tokens, Pydantic validation, parameterized ORM queries, and environment-based secrets. Error handling is explicit: Gemini errors map to 503, validation to 422, ownership failures to 404, auth failures to 401, and a global handler converts unexpected errors to a generic 500 JSON. Known gaps: no timer, no admin panel, and remaining polish items listed in Section 24.

25.4 Architecture explanation

User -> Frontend (fetch + JWT) -> FastAPI routes -> services -> Gemini (for generation/evaluation) and SQLAlchemy (for persistence) -> Pydantic serialization -> Frontend DOM. Gemini is the only external dependency and its failure is mitigated by the local question bank; the database stores everything needed to reconstruct an interview and its results.

26. Project Interview Questions

25 technical questions with model answers grounded in this repository.

#	Question	Model answer	Project file/feature
1	What is your project?	A web app for AI-based mock interviews: AI generates questions, evaluates answers, and shows scores and feedback.	Overall
2	Why did you choose this project?	Interview practice is hard to get, expensive with coaches, and generic question banks give no feedback; AI makes practice free, instant and personalized.	README, landing page
3	What is FastAPI and why use it?	A modern Python web framework for building REST APIs with automatic validation and interactive docs. Fast to develop, async-capable, and Pydantic-based validation matches the data layer.	backend/main.py
4	What is a REST API?	A style where the client and server exchange JSON over HTTP using methods like GET/POST and resource URLs.	routes
5	Which HTTP methods does the project use?	POST (register, login, create interview, get question, submit answer, complete) and GET (me, stats, interviews detail/list).	routes/*.py
6	What is JSON and where is it used?	A text format for structured data. Used for request bodies, responses, question options, and Gemini outputs.	frontend/js/api.js, schemas.py
7	What is an ORM and why SQLAlchemy?	ORM maps database tables to Python classes so we write objects instead of SQL strings. SQLAlchemy also lets SQLite and MySQL coexist.	backend/models.py, database.py
8	Explain the database tables.	users -> interviews -> questions -> answers -> feedback; all linked by foreign keys; 5 tables total.	backend/models.py, Section 8
9	What is a primary key?	A unique identifier for a row (users.id, interviews.id, ...).	backend/models.py
10	What is a foreign key?	A column referencing another table's primary key (interviews.user_id -> users.id).	backend/models.py
11	How is user authentication done?	bcrypt hashing for passwords and JWT (python-jose) for sessions; the token is sent as a Bearer header and validated by get_current_user().	backend/utils/security.py
12	Why hash passwords?	So the stored value cannot be reversed if leaked; bcrypt adds a random salt and is one-way.	backend/utils/security.py
13	Which AI provider do you use and how?	Google Gemini via its REST API, called with httpx because Python uses raw HTTP. Key in GEMINI_API_KEY, model in GEMINI_MODEL.	backend/services/gemini_service.py
14	What is a prompt?	The text instructions sent to the AI describing the task, context and output format (e.g. JSON for MCQs).	Services prompts in gemini_service.py
15	How do you make the AI return valid JSON?	Explicit instructions, a JSON-only prompt, then parse_gemini_json() with fallbacks for fences/embedded JSON, plus regex extraction and validation.	parse_gemini_json/ _normalize_mcq
16	How do you handle rate limits?	MAX_RETRIES=3 with exponential backoff honoring Retry-After, then question generation falls back to the local question bank.	_call_gemini, fallback_questions

#	Question	Model answer	Project file/feature
17	How is an MCQ scored?	The stored correct_answer letter is compared with the letter extracted from the selected option; correct=10, incorrect=0.	submit_answer/_option_letter
18	How is a written answer scored?	Gemini evaluates it and returns a JSON score 1-10 (clamped) with strengths, weaknesses, suggestions and a model answer.	evaluate_answer
19	How is the final score computed?	Per-question scores accumulate; on completion total_score becomes the average of question scores; dashboard averages completed interviews.	complete_interview, dashboard.py
20	How does the frontend talk to the backend?	fetch() calls in api.js with JSON bodies and a Bearer JWT; responses update the DOM.	frontend/js/api.js
21	How did you fix the '[object Object]' bug?	FastAPI 422 detail is an array of objects; errorMessageToString()/getErrorMessage() convert it to readable text, and the backend widened AnswerSubmit.selected_option to 500 chars.	frontend/js/api.js, schemas.py
22	What happens if Gemini is down?	Question generation falls back to the local question bank so the interview continues; written evaluation returns a readable 503 message.	fallback_questions.py, routes/interview.py
23	What is CORS and why is it configured?	CORS controls which origins a browser allows to call the API. Only localhost origins are allowed.	backend/main.py
24	Why is the difficulty system deterministic?	beginner=MCQ, advanced=writing, intermediate alternates by question number; no randomness is used, which keeps behavior predictable.	_determine_question_type
25	What are the project limitations and future work?	No timer, no admin panel, no email verification or password reset, no tests; future: timer, voice input, admin panel, Docker, tests.	README future enhancements, Sections 13/15/24

27. Final Project Summary

Item	Summary
Project	AI Interview Trainer - AI-powered mock interview training
Tech stack	Python 3.14, FastAPI, Uvicorn, SQLAlchemy, Pydantic, Google Gemini API (httpx), HTML/CSS/Vanilla JS, SQLite (SQLite/MYSQL)
Architecture	Browser frontend -> FastAPI REST API -> service layer -> Gemini + SQLAlchemy database -> Pydantic serialization -> frontend
Main features	Register/login, interview setup, AI MCQ + written questions, AI evaluation with score/strengths/weaknesses/suggestions/model answer, overall feedback, dashboard stats + history, results page, offline fallback question bank
AI integration	Google Gemini via REST; GEMINI_API_KEY/GEMINI_MODEL; retries with exponential backoff; JSON parsing with fallbacks; fallback question bank on failure
Database	SQLite interview.db in use; MySQL schema provided; tables users, interviews, questions, answers, feedback
Interview flow	Setup -> question -> answer -> evaluate -> feedback -> next -> finish -> overall feedback -> results -> history
Scoring	MCQ 10/0 by letter; writing 1-10 by AI; interview average on completion; dashboard averages; result percentage
Timer	NOT IMPLEMENTED
Admin	NOT IMPLEMENTED
Security	bcrypt password hashing, JWT bearer auth, Pydantic validation, ORM queries, CORS locked to localhost, secrets in .env (values not exposed here)
Known bugs	Previous '[object Object]' bug fixed; README drift; possible fallback repeats on long interviews; no timer/admin; see Section 24
Manual run	.\venv\Scripts\python.exe -m uvicorn backend.main:app --host 127.0.0.1 --port 8000 (or start.bat)
Manual testing	20-test plan in Section 22; timer and admin tests are N/A
Interview prep	30-sec / 2-min / 5-min explanations and 25 Q&A; in Sections 25 and 26

End of documentation. Secrets are redacted; all technical facts were verified against the source code, configuration files and the live SQLite database.